МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: И. О. Фамилия
Группа: М8О-309БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Реализовать “камеру” с возможностью ее перемещения и освещение по модели Блинн-Фонга с использованием множества типов источников света.

● Программа должна работать в реальном времени, с возможностью взаимодействия с камерой (позиция, поворот, угол обзора), источниками освещения (позиция, цвет + интенсивность, и другие параметры, уникальные для соответствующего типа источника), а также моделями (параметры материала: альбедо, цвет блика, коэффициент блика) через пользовательский интерфейс

У вершин, помимо позиции, должен быть задан атрибут нормали. Нормали вершин рисуемого объекта нужно генерировать процедурно

● Для каждого источника света должна быть реализована модель Блинн Фонга во фрагментном шейдере

● Для точечного и прожекторного источника нужно реализовать затухание по закону обратных квадратов

● Для прожекторного источника нужно реализовать мягкие края освещаемой области. Для этого нужно задать параметры внешнего и внутреннего углов

● Для доступа к массивам памяти для точечных и прожекторных источников света использовать storage буферы, чтобы не задавать максимальное количество элементов массива в шейдере

Дополнительные задания:

1. Разделите внешние данные на два набора дескрипторов: один для сцены, другой для модели. В наборе дескрипторов для сцены будут находится uniform buffer – структура с параметрами сцены и общими для всех объектов данными (матрица view-projection и пр.), а также storage буферы – массивы источников освещения. В наборе дескрипторов для модели будут находится данные о матрице трансформации и параметры матриала

2. Реализуйте освещение по модели Блинн-Фонга массивом точечных источников освещения из storage буфера, учитывайте затухание по закону обратных квадратов. В графическом интерфейсе должна быть реализована возможность добавления и удаления источников с изменением их параметров

3. Реализуйте освещение по модели Блинн-Фонга массивом прожекторных

источников освещения из storage буфера, учитывайте затухание по закону обратных квадратов, края освещаемой области должны быть мягкими, если внешний угол больше внутреннего. В графическом интерфейсе должна быть реализована возможность добавления и удаления источников с изменением их параметров.

Метод решения

Математическая часть. Для реализации камеры использована орбитальная модель в сферических координатах. Положение камеры вокруг центра сцены определяется тремя параметрами: радиусом орбиты (Distance), горизонтальным углом (Yaw) и вертикальным углом (Pitch). Координаты камеры вычисляются по формулам: X = R · cos(Pitch) · sin(Yaw), Y = R · sin(Pitch), Z = R · cos(Pitch) · cos(Yaw). Матрица вида строится через glm::lookAt с точкой взгляда в центре сцены. Угол Pitch ограничен диапазоном от -89° до +89°, чтобы избежать вырождения матрицы вида при коллинеарности направления взгляда и вектора «вверх». Из-за различий в системе координат Vulkan (ось Y в NDC направлена вниз) в вершинном шейдере дополнительно применяется инверсия gl_Position.y = -gl_Position.y.

Модель освещения Блинн-Фонга. Для каждого источника света во фрагментном шейдере вычисляется диффузная и спекулярная компоненты. Диффузная часть зависит от косинуса угла между нормалью поверхности и направлением к источнику. Спекулярная часть использует половинный вектор — нормализованную сумму направлений к источнику и к камере, что даёт более дешёвое вычисление по сравнению с классической моделью Фонга. Итоговый цвет складывается из вклада каждого источника и постоянного ambient-освещения. Нормали вершин трансформируются через inverse transpose матрицы модели, что корректно работает при неравномерном масштабировании.

Типы источников. Реализованы три типа источников света. Направленный источник задаётся вектором распространения света и освещает все точки сцены одинаково, поскольку считается бесконечно удалённым. Точечный источник задаётся позицией и

светит во все стороны с затуханием по закону обратных квадратов — интенсивность обратно пропорциональна квадрату расстояния от источника до точки. Прожекторный источник, помимо позиции и направления, имеет внутренний и внешний углы конуса: внутри внутреннего угла светит на полную яркость, снаружи внешнего — не светит, а между ними реализован плавный переход для мягких краёв. Затухание у прожектора также по закону обратных квадратов.

Объекты Vulkan. Данные разделены на два набора дескрипторов в соответствии с требованием задания. Первый набор (set 0) содержит всё общее для сцены: uniform buffer с матрицей view-projection, позицией камеры, параметрами направленного света и ambient, а также два storage buffer с массивами точечных и прожекторных источников. Второй набор (set 1) содержит данные конкретной модели: матрицу модели, inverse transpose для нормалей, альбедо, цвет блика и коэффициент блеска. Такое разделение позволяет в дальнейшем рисовать несколько объектов с общим набором сцены и разными наборами модели.

Storage buffers. Массивы источников света передаются через storage buffer, а не uniform, чтобы не задавать максимальный размер массива в шейдере. Буферы создаются с флагом VK_BUFFER_USAGE_STORAGE_BUFFER_BIT через VMA и замапливаются в память CPU для записи через memcpy. В шейдере они объявлены с ключевым словом buffer и без указания размера массива — фактическое количество активных источников передаётся через счётчик в scene uniform buffer. Структуры PointLight и SpotLight выровнены по границе 16 байт, что соответствует правилам std430.

Графический pipeline и буферы. Pipeline создаётся один раз и содержит оба шейдера, описание трёх вершинных атрибутов (позиция, нормаль, цвет), depth test и динамическое состояние viewport и scissor. Vertex и index buffers создаются через промежуточный staging buffer: данные копируются из оперативной памяти в host-visible буфер, затем командой vkCmdCopyBuffer переносятся в быструю device-local память GPU. Pipeline layout объединяет оба descriptor set layout'a и push constant для параметра

tint.

Запись команд и обновление данных. Каждый кадр в командный буфер записывается последовательность: начало render pass, установка viewport и scissor, привязка pipeline, привязка двух descriptor set'ов, передача push constants, привязка vertex и index buffer, вызов vkCmdDrawIndexed для отрисовки сферы. В функции обновления каждый кадр заполняются структуры scene и model из параметров UI, затем через memcpy копируются в соответствующие uniform и storage buffers.

Результаты

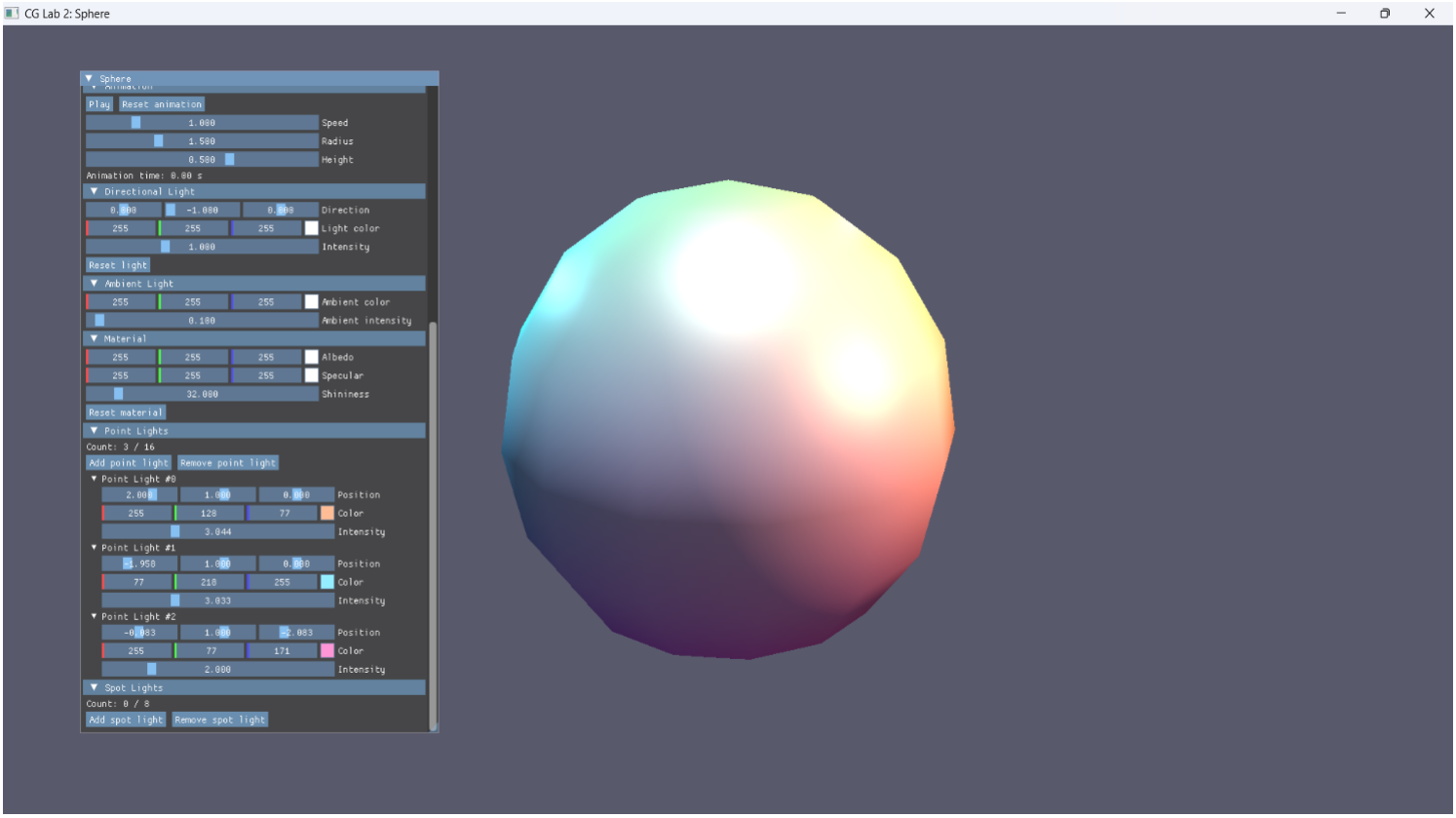


Рисунок 1. Освещение сцены массивом точечных источников через storage buffer. На поверхность сферы проецируются три цветных пятна (оранжевое, голубое, розовое) — вклады каждого источника в общий цвет пикселя по модели Блинн-Фонга. Интенсивность каждого источника затухает по закону обратных квадратов, что видно по плавному спаду яркости от центра пятна к периферии.

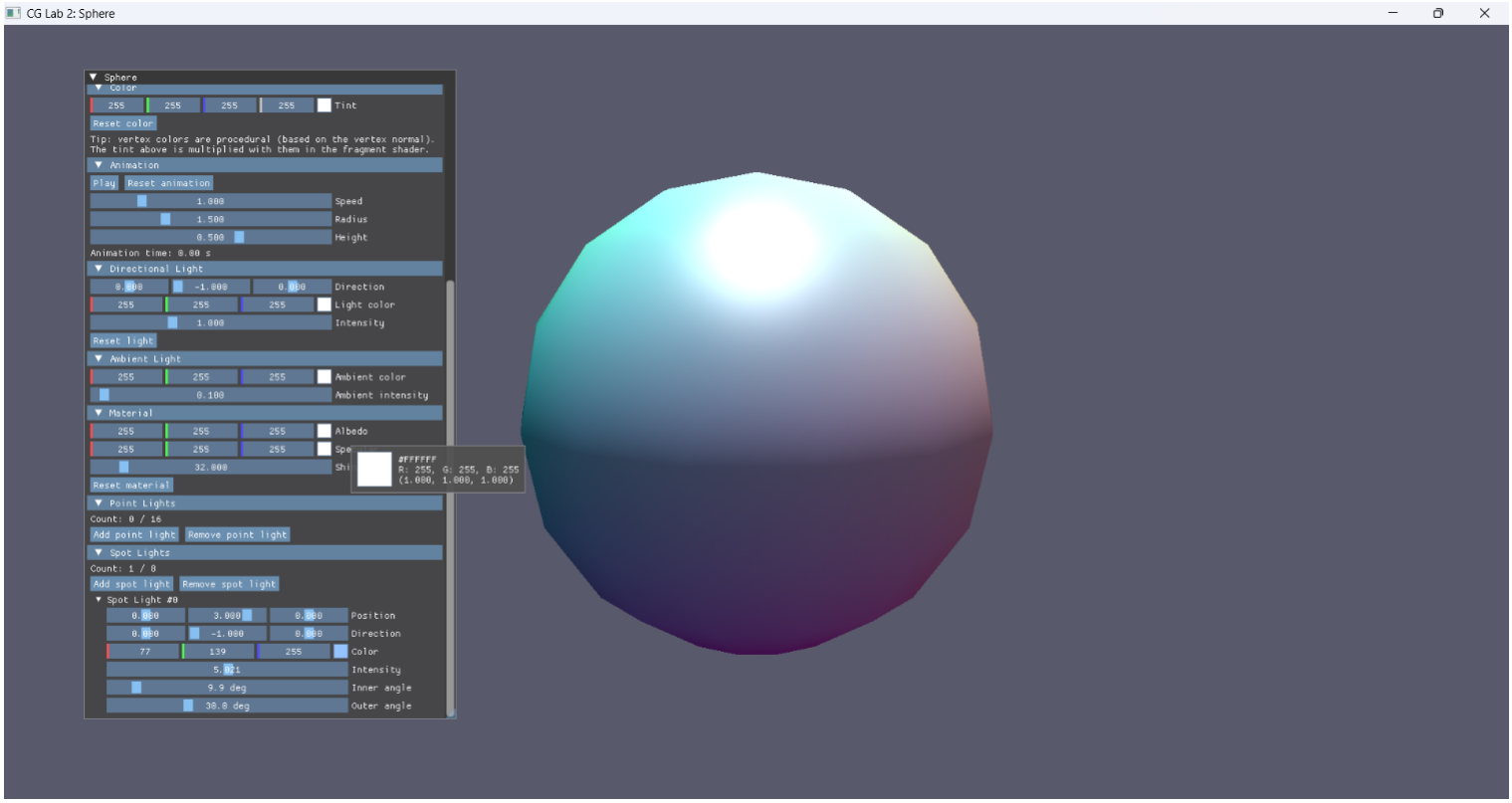


Рисунок 2. Прожекторный источник света с мягкими краями. Узкий конус с плавным переходом яркости от центра к краю. Коэффициент spot_factor рассчитывается как clamp((theta - cos(outer)) / (cos(inner) - cos(outer)), 0, 1), где theta — косинус угла между направлением прожектора и лучом от источника к точке.

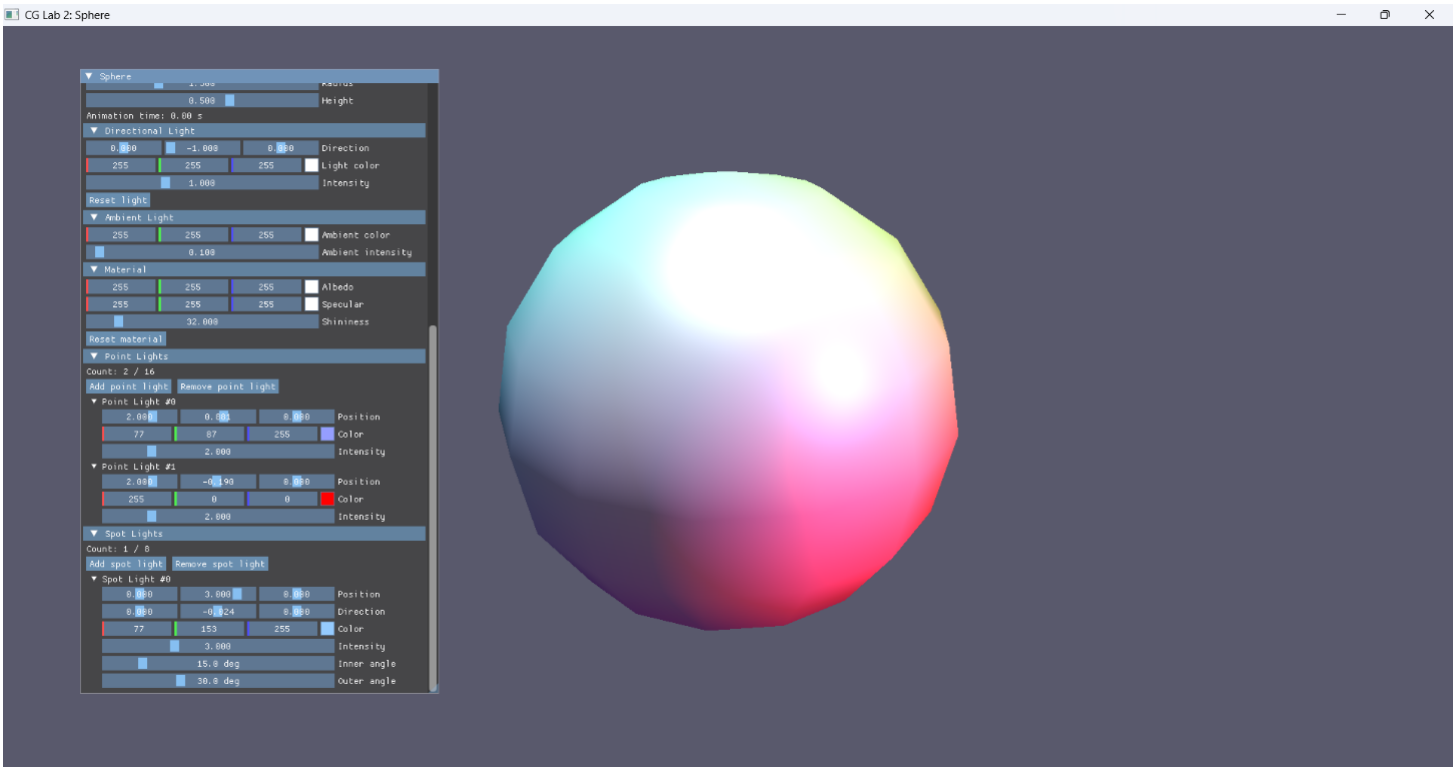


Рисунок 3. Комбинированное освещение сцены тремя типами источников:

направленным, двумя точечными и одним прожекторным. Каждый источник вносит независимый вклад в освещение, все параметры (позиции, цвета, интенсивности, углы) меняются в реальном времени без перезапуска программы.

Выводы

В ходе выполнения лабораторной работы я освоил ключевые концепции современной компьютерной графики и Vulkan API. Была реализована полноценная сцена с динамическим освещением по модели Блинн-Фонга с тремя типами источников света — направленным, точечным и прожекторным — и орбитальной камерой, управляемой из пользовательского интерфейса ImGui в реальном времени.